TechNest – Front-End eCommerce Web Application

Week 2 Planning Document

Muhammad Saleem | L00196822 | ATU

Week: 2 of 10

This document covers the planning and implementation work completed in Week 2 of the TechNest project. It includes the two structural diagrams produced this week, the layered architecture diagram and the class diagram, along with the HTML structure built for all five pages, Bootstrap integration decisions and a summary of what was completed.

---

1. Structural Diagrams

Two structural diagrams were produced this week to give a clear visual picture of how the application is organised before any JavaScript logic was written. These build on the architecture plan and class descriptions from Week 1.

1.1 Layered Architecture Diagram

The architecture diagram shows TechNest organised into four layers stacked from top to bottom.

Presentation Layer contains the five HTML pages and the shared style.css and Bootstrap 5, which handle layout and styling across every page.

Logic Layer contains the page-specific JavaScript modules in js/pages/ and the two ES6 class files, Basket.js and Product.js, which hold the core application logic. Each page script has one job – for example, products.js handles rendering the catalogue, filtering and search, while basket.js handles displaying basket items and updating quantities.

Data Layer contains StorageManager.js for reading and writing to localStorage, and the products.js data file in js/data/ which is a static array of product objects and acts as the single source of truth for all product information. There is no database or server – this is an intentional design decision.

Browser Runtime Layer at the bottom shows the three browser APIs the application depends on: the localStorage API for persisting basket data, the DOM API for dynamically rendering products and basket contents, and the URL/Location API for reading URL parameters on the product detail page.

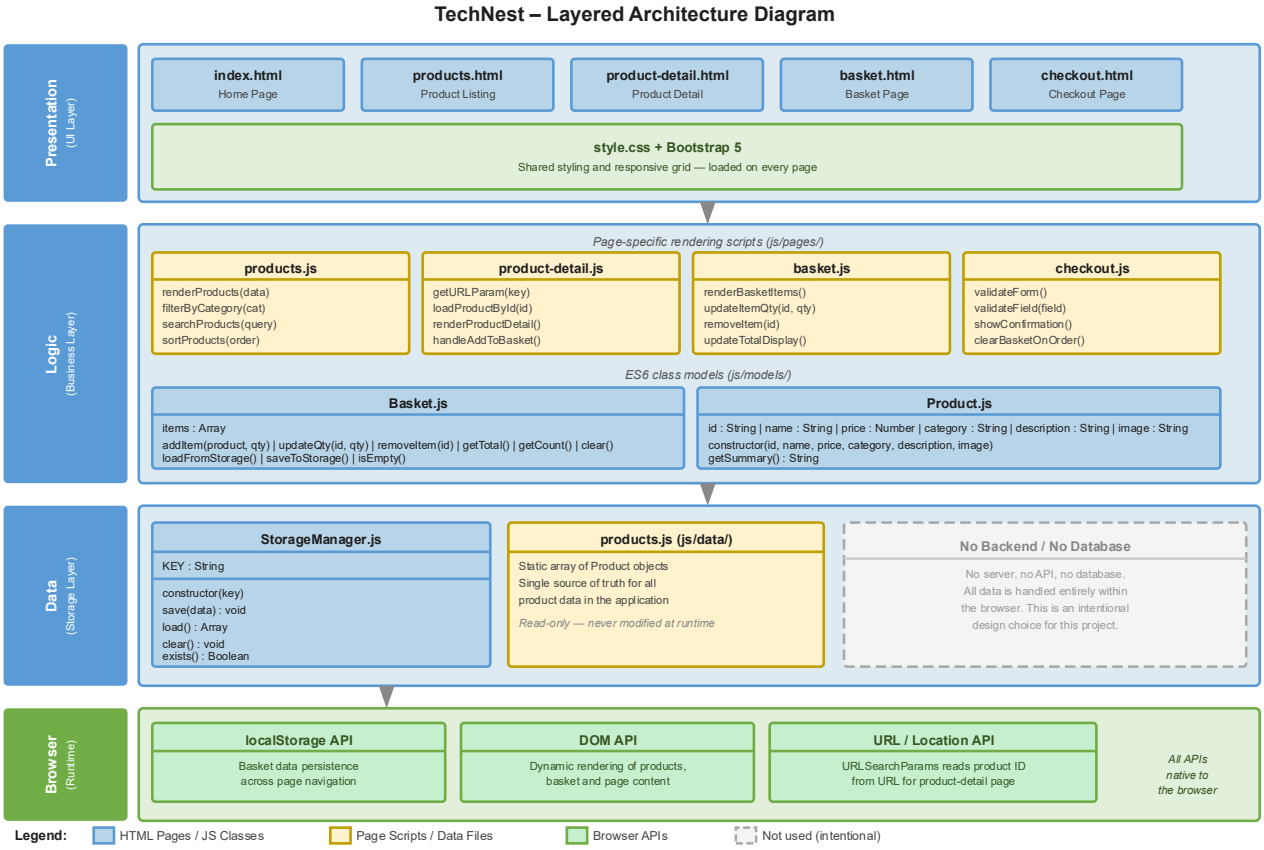

Figure 1: Layered Architecture Diagram

1.2 Class Diagram

The class diagram shows the three ES6 classes and how they relate to each other.

Product holds six properties – id, name, price, category, description and image – plus a constructor and a getSummary() method. It is a straightforward data class with no dependencies on the other two classes.

Basket is the most involved class. It holds an items array and has methods for addItem(), updateQty(), removeItem(), getTotal(), getCount(), clear(), loadFromStorage() and saveToStorage(). Product objects are stored inside the Basket items array, shown by a solid arrow from Product to Basket in the diagram.

StorageManager handles all localStorage interaction. It has a KEY property and five methods: constructor(key), save(), load(), clear() and exists(). The dashed arrow from Basket to StorageManager shows the dependency relationship – Basket calls StorageManager to persist and retrieve its state.

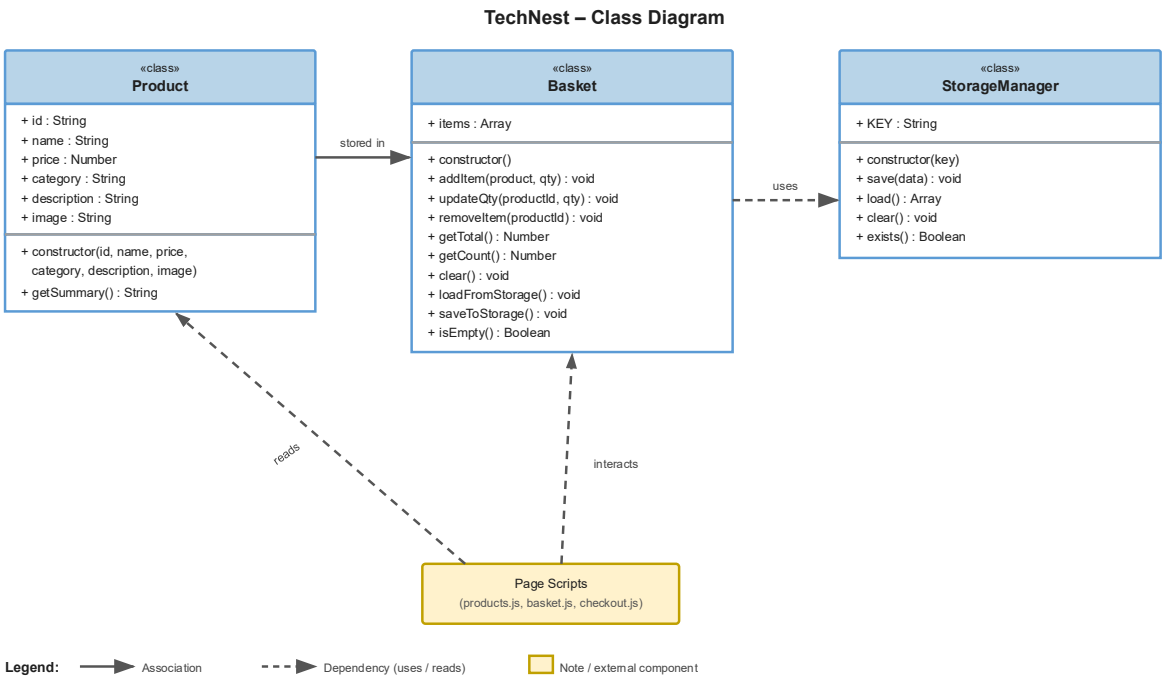

Figure 2: Class Diagram

---

2. HTML Structure

All five HTML pages were built this week using semantic HTML5 elements. Each page shares the same navigation bar and footer. No custom styling or JavaScript was added at this stage – the focus was purely on getting the correct semantic structure in place before styling and logic are added in later weeks.

2.1 Shared Elements

A shared navigation bar was built and added to all five pages. It contains:

- TechNest brand name on the left, linking back to index.html
- A Products link in the navigation
- A Basket icon on the right with a count badge (set to 0 as a placeholder)

A shared footer was also added to each page containing the project name and student number.

2.2 index.html – Home Page

Semantic structure:

```
<header> <nav> shared navbar </nav> </header>
<main>
  <section class="hero"> heading, description, Shop Now button </section>
  <section class="categories"> category filter buttons </section>
  <section class="featured-products"> placeholder for product cards </section>
</main>
<footer> shared footer </footer>
```

2.3 products.html – Product Listing

Semantic structure:

```
<header> <nav> shared navbar </nav> </header>
<main>
  <section class="filters">
    <input type="search">  keyword search input
    <select>  sort by price dropdown
    <div class="category-buttons">  category filter buttons
  </section>
  <section class="product-grid">  placeholder for rendered cards </section>
</main>
<footer> shared footer </footer>
```

2.4 product-detail.html – Product Detail

Semantic structure:

```
<header> <nav> shared navbar </nav> </header>
<main>
  <a href="products.html">  Back to Products link
  <section class="product-detail">
    <div class="product-image">  large product image placeholder
    <div class="product-info">
      product name heading, category badge, description,
      price, quantity input, Add to Basket button
    </div>
  </section>
</main>
<footer> shared footer </footer>
```

2.5 basket.html – Basket Page

Semantic structure:

```
<header> <nav> shared navbar </nav> </header>
<main>
  <section class="basket-layout">
    <div class="basket-items">  empty placeholder for basket rows
    <div class="order-summary">  subtotal, Proceed to Checkout button
  </section>
  <p class="empty-basket" hidden>  Your basket is empty message
</main>
<footer> shared footer </footer>
```

2.6 checkout.html – Checkout Page

Semantic structure:

```
<header> <nav> shared navbar </nav> </header>
<main>
  <section class="checkout-layout">
```

```
    <form id="checkout-form">
      <fieldset>  Personal Details: first name, last name, email
      <fieldset>  Delivery Address: address lines, city, postcode
      <fieldset>  Payment Details: card number, expiry, CVV
      <button type="submit">  Place Order
    </form>
    <div class="order-summary">  basket summary and total
  </section>
  <div class="confirmation" hidden>  order confirmation message
</main>
<footer> shared footer </footer>
```

---

3. Bootstrap Integration

Bootstrap 5 was linked via CDN in the <head> of every page this week. No custom CSS was added yet. The following Bootstrap classes were used in the HTML structure to get the basic responsive layout in place:

Class	Purpose
container	Centres content with consistent side margins on all pages
row / col	Grid layout for two-column pages such as detail, basket and checkout
navbar / navbar-expand-lg	Responsive navigation bar that collapses on smaller screens
btn / btn-primary	Styled buttons used for Shop Now, Add to Basket and Place Order
badge	Basket item count displayed in the navbar
card	Product card structure on the products page
form-control	Styled form inputs across the checkout page

---

4. Decisions Made This Week

Semantic HTML over divs

Semantic elements such as <main>, <section>, <header>, <footer> and <fieldset> were used throughout rather than plain elements wherever the element has a meaningful role on the page. This makes the structure easier to read and understand without relying on class names to explain what each block does.

Shared navbar copied manually

The navbar is copied and pasted into each HTML file rather than using any templating system. Since TechNest has no backend there is no way to include shared components automatically. This means if the navbar changes it needs updating in all five files, which is acceptable for a five-page project.

Confirmation div hidden with the hidden attribute

On the checkout page the order confirmation message is placed in the HTML but hidden using the HTML hidden attribute. JavaScript will show it and hide the form when the order is placed successfully. Keeping it in the HTML rather than creating it dynamically makes it easier to style and test.

Empty basket message hidden by default

Similarly on the basket page the empty basket message is present in the HTML but hidden. JavaScript will toggle its visibility depending on whether the basket has any items in it.

---

5. Week 2 Summary

The following tasks were completed during Week 2:

- Produced the layered architecture diagram showing the four-layer structure of TechNest
- Produced the class diagram showing Product, Basket and StorageManager and their relationships
- Built semantic HTML structure for all five pages
- Set up the shared navigation bar and footer across all pages
- Linked Bootstrap 5 via CDN on every page and applied basic layout classes
- Committed all HTML structure to GitHub with a meaningful commit message

Next week (Week 3) the focus will be on adding custom CSS styling, creating the products data file and rendering the product cards dynamically on the products page.